cbToy Davidson Solver

MPI scaling, k-point pools, and GPU offload

March 13, 2026

Core message

cbToy is a compact, educational plane-wave eigensolver whose performance story is clear: FFT-based Hamiltonian application scales reasonably, the reduced eigensolve becomes the CPU bottleneck, k-point pools add near-ideal parallelism, and GPU offload cuts time-to-solution dramatically.

Problem Setup

Physical model

Solve the plane-wave Kohn–Sham eigenproblem

$$H|\psi_n\rangle = \varepsilon_n S|\psi_n\rangle$$

in a plane-wave basis using the Cohen–Bergstresser pseudopotential for FCC diamond/zincblende semiconductors.

Numerical ingredients

- Plane-wave basis truncated by `ecutwfc`
- Matrix-free Hamiltonian: kinetic term plus local potential via FFTs
- Block Davidson iteration for the lowest `nbnd` eigenpairs

Benchmarks used in the report

- **CPU:** `si4.in`
- **k-point pools:** `si2_11_points.in`
- **GPU:** `si4_80_cut.in`
- **GPU:** `si2_11_points_30_cut.in`

How the Solver Works

Main workflow

1. Read input, generate G-vectors, and build the local potential
2. For each k-point, initialize trial wavefunctions
3. Run `cegterg` or `pcegterg` to convergence
4. Write band energies, optionally shifted to the valence-band maximum

Three code variants

- `cb_davidson_mpi.f90`: CPU MPI baseline
- `cb_davidson_mpi_kp.f90`: parallelism on k-point pools
- `cb_davidson_mpi_kp_acc.f90`: OpenACC offload

Dominant kernel

Every Davidson step repeatedly evaluates $H\psi$:

$$H\psi = \frac{1}{2}|k+G|^2\psi + \mathcal{F}\left[V_{\mathrm{loc}}(r)\,\mathcal{F}^{-1}\{\psi\}\right]$$

This costs two 3D FFTs per band, which makes FFT efficiency central.

Parallelization Hierarchy

Communicator tree

```
MPI_COMM_WORLD
  |- k-point pools       (-nk)
  |    |- band groups    (-nband)
  |    \- diag groups    (-ndiag)
  \- inter-pool gather
```

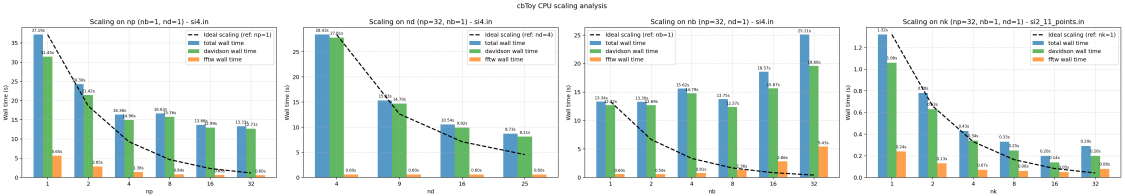
What is distributed?

- Plane waves / FFT grid within each pool
- Bands across band groups
- Reduced matrices across a BLACS 2D grid when `ndiag > 1`
- Entire k-points across pools in the extended code

Performance speedup

- **PW / FFT:** distributes memory, but pays all-to-all transposes
- **Band groups:** lowers per-group band work, but can starve FFT parallelism
- **Distributed diag:** attacks the reduced eigensolve once FFTs are already fast
- **k-point pools:** almost ideal because k-points are independent during the solve

CPU MPI Scaling on One Node



Findings

PW scaling helps FFT time, band groups mostly do not help for this problem size, and the best CPU result comes from distributed diagonalization:

$$(np, nb, nd) = (32, 1, 25) \Rightarrow 3.95 \text{ s}$$

compared with a **32.02 s** serial baseline.

Why CPU Scaling Is Far From Ideal

1. Amdahl bottleneck

With `nd = 1`, the reduced eigenproblem and subspace projections remain the dominant non-scalable share.

Report estimate:

$$f \approx 0.81$$

so PW parallelism alone cannot deliver close-to-linear scaling.

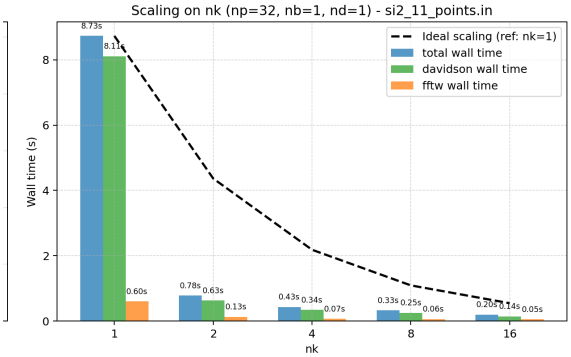
2. FFT communication

Each 3D FFT needs data transposes across ranks. Even when local FFT work shrinks, the all-to-all communication cost and mild load imbalance remain.

3. Small-ish problem

The Si 4^3 case has only about 6.6×10^3 plane waves and 512 bands, so 32 ranks push the run toward a latency-dominated regime.

K-Point Pools Add the Cleanest Extra Parallelism



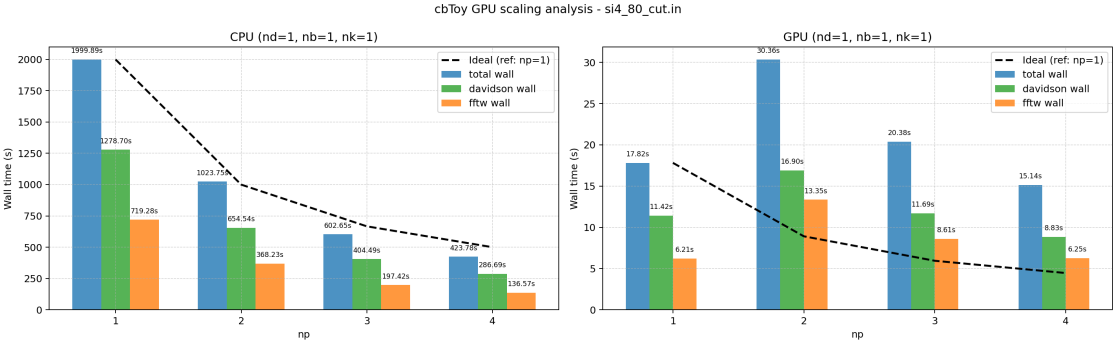
Pool idea

`cb_davidson_mpi_kp.f90` splits processes into pools; each pool solves only its assigned k-points and sends eigenvalues back once at the end with `MPI_Gatherv`.

Measured effect

- Best point: `nk` = 16 on 32 MPI ranks
- Davidson wall time drops from **1.06 s** to **0.14 s**
- Speedup is **7.6x** because the 11 k-points are independent during the solve
- `nk` = 32 is worse than `nk` = 16 because only 11 pools have work

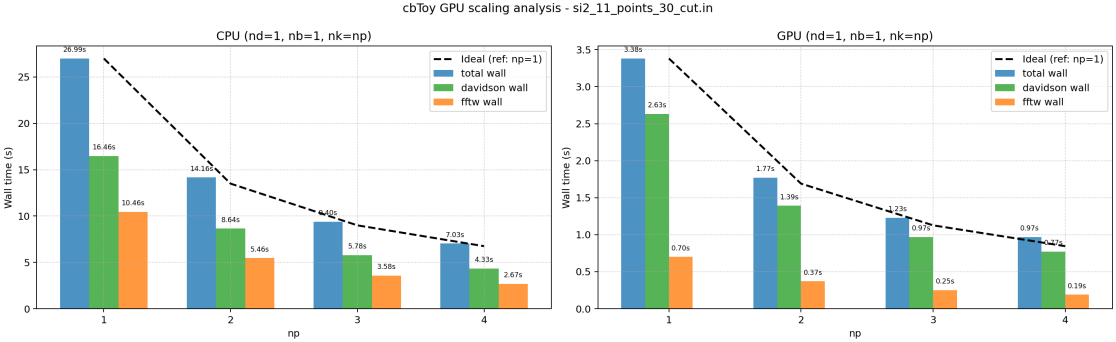
GPU Offload Changes Time-to-Solution More Than Strong Scaling



Findings

The OpenACC version keeps the same MPI structure but offloads the hot kernels. The biggest benefit is absolute time-to-solution: **32.47x** CPU/GPU speedup at `np = 4`.

GPU K-Point Pools Scale Much Better



Findings

This experiment switches from single-k-point strong scaling to `nk=np` on `si2_11_points_30_cut.in`, so every added rank/GPU also adds a k-point pool. That removes most communication during the solve and makes the GPU path scale much more cleanly: total wall time drops from **3.38 s** on 1 GPU to **0.97 s** on 4 GPUs, while the GPU run stays about **7–8x** faster than the CPU reference across the sweep.

Summary

Findings

- On CPU, distributed diagonalization is the most important improvement once FFTs are already parallelized.
- k-point pools are the cleanest extra scaling axis because they avoid communication during the solve.
- GPU offload delivers the biggest absolute speedup, but strong scaling still depends on keeping communication overhead under control.